

Generics

→ Generics offer the ability to have one class which is flexible for different types

```
public class Printer <T> {
```

→ What this <T> represents is the type of thing that this printer will be able to hold & print

```
package org.kdklearns.generics;

public class Printer <T> { no usages
    T thingToPrint; 2 usages

    public Printer(T thingToPrint) { no usages
        this.thingToPrint = thingToPrint;
    }

    public void print() { no usages
        System.out.println(thingToPrint);
    }
}
```

→ Generics don't work with primitive types like int, long etc.

→ Therefore, we must use wrapper classes instead

→ Hence, Generics help us by offering the ability to create classes that can accommodate lots of different types, but also a structure and the type safety of knowing exactly which type of class you're using at the moment

Bounded Generic

→ We give the certain bound a limit on the type that can pass in

```
public class Printer < T extends Animal > {
```

→ Now this T has access to the behaviour & states of Animal

Generic Methods

→ If you want to make a method generic, such that it can accept, print etc. any type T

```
private static < T > void shout ( T thingToShout ) {
```

→ Generic methods can also take multiple generic parameters as type

```
public static < K, V > void shout ( K thingToShout, V otherThingToShout )
```

→ Similar to methods, even classes can hold / take multiple types. for example →

```
Map < String, Integer > map = new HashMap < > ();
```